

Informe Final de Hardening y Seguridad

Este documento consolida el trabajo de seguridad y hardening realizado sobre la arquitectura de microservicios del proyecto. Cada sección detalla la implementación, el estado y las evidencias para su verificación.

2.1. HTTPS/TLS (Comunicación Segura)

Estado: ☒ Completado

Requisitos

- El Frontend debe ser accesible únicamente por HTTPS.
- El API Gateway debe tener un certificado TLS configurado con redirección automática de HTTP a HTTPS.
- Se deben implementar Headers de seguridad HTTP (HSTS, X-Frame-Options, CSP, etc.).
- Se permite el uso de certificados autofirmados para el entorno de desarrollo.

Implementación

Se ha implementado HTTPS/TLS en el API Gateway (Nginx) para asegurar toda la comunicación externa.

1. **Generación de Certificados:** Se utiliza un script para generar certificados autofirmados (`cert.pem` y `key.pem`) para `dev.local` y `127.0.0.1`, válidos para el entorno de desarrollo.
2. **Configuración del API Gateway (`services/api-gateway/nginx.conf`):**
 - **SSL/TLS:** El puerto 443 está configurado para usar los certificados generados, restringiendo los protocolos a versiones seguras (TLSv1.2 y TLSv1.3) y utilizando un conjunto de cifrados robustos.
 - **Redirección:** El puerto 80 (expuesto en el host como 8080) realiza una redirección 301 permanente a la versión HTTPS en el puerto 443 (expuesto como 8443).
 - **Headers de Seguridad:** Se han añadido los siguientes encabezados para mitigar ataques comunes del lado del cliente:
 - **Strict-Transport-Security (HSTS):** Obliga a los clientes a comunicarse únicamente a través de HTTPS.
 - **X-Frame-Options:** Previene ataques de Clickjacking.
 - **X-Content-Type-Options:** Evita que el navegador interprete archivos con un tipo MIME diferente al declarado.
 - **Content-Security-Policy (CSP):** Controla qué recursos puede cargar el navegador y fuerza la actualización de peticiones inseguras (`upgrade-insecure-requests`).

Evidencia y Verificación

1. Generar los certificados:

```
# Ejecutar desde la raíz del proyecto con wsl
wsl sudo bash ./scripts/security/generate-dev-cert.sh
```

Esto creará los archivos `cert.pem` y `key.pem` en `services/api-gateway/certs/`.

2. Verificar la redirección HTTP a HTTPS:

```
curl.exe -k -I http://dev.local:8080/
```

Salida esperada:

```
HTTP/1.1 301 Moved Permanently
Location: https://dev.local:8443/
...
```

3. Verificar los Headers de Seguridad:

```
curl.exe -k -I https://dev.local:8443/
```

Salida esperada (extracto):

```
HTTP/1.1 403 Forbidden
...
Strict-Transport-Security: max-age=31536000; includeSubDomains
X-Frame-Options: SAMEORIGIN
X-Content-Type-Options: nosniff
Content-Security-Policy: default-src 'self'; ... upgrade-insecure-requests;
...
```

Al aplicar el siguiente comando omitimos configuración de WAF

```
curl.exe -k -I -A "Mozilla/5.0" https://dev.local:8443/
```

Salida esperada (extracto):

```
HTTP/1.1 200 OK
...
Strict-Transport-Security: max-age=31536000; includeSubDomains
X-Frame-Options: SAMEORIGIN
X-Content-Type-Options: nosniff
Content-Security-Policy: default-src 'self'; ... upgrade-insecure-requests;
...
```

4. **Acceder desde el navegador:** Visita <https://dev.local:8443>. Deberás aceptar el riesgo del certificado autofirmado. Una vez cargada la página, el navegador mostrará un candado, indicando que la conexión es HTTPS.
-

2.2. Hardening de Contenedores Docker

Estado: ☒ Completado (con excepciones documentadas)

Requisitos

- Ejecutar contenedores con usuarios no privilegiados (no **root**).
- Usar imágenes base con versiones específicas (evitar **:latest**).
- Utilizar *multi-stage builds* para reducir la superficie de ataque.
- Configurar **security_opt: no-new-privileges** y **cap_drop: [ALL]**.
- Escanear imágenes en busca de vulnerabilidades.

Implementación

1. **Usuario no privilegiado:** Implementado en todos los servicios de backend (Node.js) y en la pila de monitoreo. Se crea un usuario **node** en los Dockerfiles correspondientes.
2. **Imágenes base específicas:** Todos los Dockerfiles utilizan imágenes con versiones explícitas, como **node:18.18.2-alpine**.
3. **Multi-stage builds:** Implementado en los servicios Node.js, separando el entorno de compilación del de ejecución para obtener una imagen final más ligera y segura.
4. **Opciones de seguridad en Docker Compose:** Se han añadido las directivas **security_opt: [no-new-privileges]** y **cap_drop: [ALL]** a los servicios de backend y monitoreo para limitar los privilegios de los contenedores.
5. **Excepción documentada (Nginx):** Los contenedores del API Gateway y los frontends (que usan Nginx) se ejecutan como **root**. Esto se debe a que la imagen oficial de Nginx intenta cambiar la propiedad de directorios de caché (**/var/cache/nginx**) en su arranque. Esta operación falla si el contenedor se ejecuta con **no-new-privileges**, causando un bucle de reinicio. Esta es una mitigación temporal para asegurar la funcionalidad.

Evidencia y Verificación

- **Revisar Dockerfiles:** Inspeccionar los **Dockerfile** en las carpetas de servicios como **ai-service**, **auth-service**, etc., para verificar el uso de **USER node** y los *multi-stage builds*.
- **Revisar docker-compose.yml:** Confirmar la presencia de las claves **security_opt** y **cap_drop** en la definición de los servicios.

Análisis de Vulnerabilidades

Estado: ☒ Completado

- Se utiliza docker scout usandolo a través del script **generarMarkdown.js**, para esto se le pasa el listado de imágenes a analizar con un **imagenes.txt** y esto genera un **.md** con el CVE, la Severity y el total de vulnerabilidades de las imágenes.
-

2.3. Hardening de Base de Datos

Estado: ☒ Completado

Requisitos

- Autenticación obligatoria con contraseñas fuertes (>12 caracteres).
- Usuario de aplicación con privilegios mínimos (solo CRUD).
- Aislamiento de red (sin exponer puertos al host).
- Logging de conexiones y actividad.

Implementación

1. **Aislamiento de Red:** El servicio `postgres-master` no tiene puertos expuestos al host. Solo es accesible a través de la red interna `database-network` por los servicios autorizados (como `db-proxy`).
2. **Autenticación Robusta:** Se utiliza `scram-sha-256` como método de autenticación. Las credenciales se gestionan como variables de entorno.
3. **Privilegios Mínimos:**
 - Se crea un rol de aplicación específico (`APP_DB_USER`).
 - Este rol NO tiene permisos de superusuario, creación de roles o bases de datos.
 - Solo se le conceden los permisos `SELECT`, `INSERT`, `UPDATE`, `DELETE` sobre el esquema `public`.
4. **Logging y Auditoría:** Se habilitó el registro de conexiones, desconexiones, duraciones de sentencias e intentos fallidos. Los logs se emiten a `stderr` para ser capturados por el sistema centralizado (Loki).

Evidencia y Verificación

1. **Verificar que el puerto no está expuesto:**

```
docker compose ps
```

Salida esperada: La columna `Ports` para el servicio `postgres-master` debe estar vacía o mostrar solo el puerto interno (`5432/tcp`), sin mapeo al host.

2. **Intentar conexión desde el host (debe fallar):**

```
# Asume que tienes psql instalado localmente
psql -h localhost -p 5432 -U user -d municipalidad_db
```

Salida esperada: `psql: error: connection to server at "dev.local" (127.0.0.1), port 5432 failed: Connection refused Is the server running on that host and accepting TCP/IP connections?.`

3. **Verificar conexión interna a través del proxy (debe funcionar):**

```
# Ejecuta un comando psql dentro de un contenedor que tenga acceso a la red de la BD
```

```
docker exec -e PGPASSWORD=user123_adminX -it postgres-master psql -h db-proxy -p 5432 -U user -d municipalidad_db -c "SELECT 1;"
```

Salida esperada: (1 row)

4. Consultar logs de intentos fallidos:

```
docker logs postgres-master | grep "FATAL"
```

Esto mostrará los intentos de conexión rechazados por `pg_hba.conf`.

2.4. Gestión de Secretos

Estado: ☒ Completado

Requisitos

- No "hardcodear" secretos en el código o en `docker-compose.yml`.
- Utilizar un archivo `.env` que esté ignorado por Git.
- Proporcionar una plantilla `env.example`.
- Implementar un script para la rotación de secretos.

Implementación

- Se utiliza un archivo `.env` en la raíz del proyecto para definir todas las credenciales y secretos. `docker-compose` carga estas variables automáticamente.
- El archivo `.env` está incluido en `.gitignore` para prevenir su publicación accidental.
- Se proporciona un archivo `.env.example` como plantilla para los usuarios.
- Se creó el script `scripts/security/rotate-secrets.sh` que genera nuevas credenciales seguras y las formatea para ser añadidas al archivo `.env`.

Evidencia y Verificación

1. **Verificar `.gitignore`:** Confirma que la línea `.env` existe en el archivo `.gitignore`.
2. **Inspeccionar `.env.example`:** Revisa que este archivo contenga las claves de las variables de entorno pero con valores de ejemplo o vacíos.

Estado: ☐ Pendiente No se implementó rotate secrets.

2.5. WAF / Rate Limiting

Estado: ☒ Completado

Requisitos

- Implementar un WAF (Web Application Firewall) a nivel de Gateway.
- Limitar la tasa de peticiones por IP (`Rate Limiting`).

- Bloquear **User-Agents** maliciosos conocidos.
- Registrar los eventos de bloqueo.

Implementación

El API Gateway (Nginx) ha sido configurado para actuar como un WAF básico.

1. **Rate Limiting:** Se ha configurado una zona de memoria (**perip**) que rastrea las peticiones por dirección IP. Se aplica un límite de **10 peticiones por segundo** con una ráfaga (**burst**) de 20 en todas las rutas bajo **/api/**. Las peticiones que superan este límite reciben un error **429 Too Many Requests**.
2. **Bloqueo de User-Agents:** Se ha implementado una regla que inspecciona el **User-Agent** de cada petición. Si coincide con patrones maliciosos conocidos (ej., "sqlmap"), la petición es denegada con un error **403 Forbidden**.

Evidencia y Verificación

1. Probar bloqueo de User-Agent (403):

```
curl.exe -k -I -A "sqlmap" https://dev.local:8443/api/auth/
```

Salida esperada:

```
HTTP/1.1 403 Forbidden
...
```

En los logs de Loki, este evento aparecerá con la etiqueta **blocked_ua=1**.

2. Probar Rate Limiting (429):

```
# Lanza una ráfaga de 200 peticiones
1..200 | % { curl.exe -k -s -o /dev/null -w "%{http_code}\n" -A "Mozilla"
https://localhost:8443/api/auth/ }
```

Salida esperada: Verás una mezcla de códigos de respuesta, incluyendo **404** (si el endpoint no existe) y **429**, confirmando que el límite se activó.

3. Consultar logs de bloqueo:

```
# Usando el script de consulta
sh ./scripts/security/query-logs.sh '{job="security-gateway"}' |= "limiting
requests"
```

Esto mostrará los logs de Nginx donde se registra la activación del **rate limiting**.

2.7. Logging y Auditoría

Estado: ☒ Completado

Requisitos

- Implementar un sistema de logging centralizado.
- Registrar eventos de seguridad clave: autenticación, accesos denegados (401/403), acceso a endpoints sensibles y conexiones a la base de datos.

Implementación

- Se utiliza la pila **Loki + Promtail + Grafana** para la recolección, almacenamiento y visualización de logs de todos los contenedores.
- **Configuración de Promtail:** El archivo `promtail-config.yml` define cómo se recolectan y etiquetan los logs:
 - Los servicios tienen una etiqueta `logging_jobname` en `docker-compose.yml`.
 - Los logs del gateway se etiquetan como `job=security-gateway` y se parsean para extraer información relevante de los accesos.
 - Los logs de las aplicaciones se etiquetan como `job=security-app`. Se recomienda que las aplicaciones emitan logs en formato JSON para un parseo y filtrado más eficiente.
- **Eventos registrados:**
 - **Gateway:** Accesos, errores, bloqueos de WAF (403), y rate limiting (429).
 - **Base de Datos:** Conexiones, desconexiones y errores de autenticación.
 - **Aplicaciones:** Intentos de login, errores de autorización y otros eventos de negocio (requiere instrumentación en el código de cada servicio).

Evidencia y Verificación

1. **Acceder a Grafana:** Abre `http://localhost:3010` en tu navegador. El usuario y contraseña por defecto son `admin/admin`.
2. **Explorar logs en Grafana:**
 - Navega a la sección "Explore".
 - Selecciona "loki" como fuente de datos.
 - Usa consultas LogQL para filtrar los logs.
3. **Ejemplos de Consultas LogQL:**
 - Bloqueos por WAF en el gateway: `{job="security-gateway"} |= "403"`
 - Activación de Rate Limiting: `{job="security-gateway"} |= "limiting requests"`
 - Logs de la base de datos: `{container_name="postgres-master"}`
 - Logs del servicio de autenticación: `{job="security-app", service="auth-service"}`
4. **Usar el script de consulta:**

```
sh ./scripts/security/query-logs.sh '{job="security-gateway"} |= "403"'
```

Este script proporciona una forma rápida de consultar los logs desde la terminal.

2.8. Políticas y Documentación

Estado: ☒ Completado

Los siguientes documentos se adjuntan en /docs

- docs/politica-seguridad.md
- docs/matriz-riesgos.md
- docs/owasp-top10.md
- docs/plan-incidentes.md
- docs/cumplimiento-normativo.md